

Numerical Addendum: HBond Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

HBond is the McConnell dyadic family evaluated on DSSP-resolved hydrogen-bond sources. The shared McConnell addendum treats the three-term asymmetric tensor, the scalar f , the symmetric-traceless dipolar helper, and the $T_0/T_1/T_2$ projection; this addendum records what changes here: the source list comes from DSSP partner slots, the source point is the $N \cdots O$ midpoint, the source axis is the donor- N to acceptor- O direction, and filter tests decide which atom-source pairs enter the sum. The listings below are distilled from the source: loops and bookkeeping are trimmed, but the constants, guards, inequalities, and output shapes are the implementation's.

1 Resolving DSSP pairs

DsspResult gives each residue two acceptor slots and two donor slots. HBond reads those residue indices; it does not read the DSSP energies when it forms the source list. On the acceptor-slot pass, the current residue supplies the donor backbone N and the partner residue supplies the acceptor backbone O . On the donor-slot pass, the partner residue supplies the donor N and the current residue supplies the acceptor O . Both passes feed the same resolution test, and the accepted object is the ordered atom pair $(N_{\text{donor}}, O_{\text{acceptor}})$.

```
// Distilled from HBondResult::Compute, source resolution.
auto add_pair = [&](size_t donor_ri, size_t acceptor_ri) {
    if (donor_ri == SIZE_MAX || acceptor_ri == SIZE_MAX) return;
    if (donor_ri >= n_residues || acceptor_ri >= n_residues) return;

    const Residue& donor = protein.ResidueAt(donor_ri);
    const Residue& acceptor = protein.ResidueAt(acceptor_ri);
    if (donor.N == Residue::NONE || acceptor.O == Residue::NONE) return;

    int seq_sep = std::abs(static_cast<int>(donor_ri)
        - static_cast<int>(acceptor_ri));
    if (seq_sep < cfg("hbond_sequential_exclusion_residues")) return; // 2

    auto key = std::make_pair(donor.N, acceptor.O);
    if (seen.count(key)) return;
    seen.insert(key);

    Vec3 N_pos = conf.PositionAt(donor.N);
    Vec3 O_pos = conf.PositionAt(acceptor.O);
    Vec3 axis = O_pos - N_pos;
    double L = axis.norm();
    if (L < cfg("singularity_guard_distance") ||
        L > cfg("hbond_dipolar_max_distance")) return; // 50 Å

    ResolvedHBond hb;
    hb.donor_N = donor.N;
    hb.acceptor_O = acceptor.O;
    hb.midpoint = 0.5 * (N_pos + O_pos);
    hb.h_hat = axis / L;
    hb.distance = L;
    honds.push_back(hb);
};

for (size_t ri = 0; ri < n_residues; ++ri) {
    for (int slot = 0; slot < 2; ++slot) {
        add_pair(ri, dssp.AllResidues()[ri].acceptors[slot].residue_index);
        add_pair(dssp.AllResidues()[ri].donors[slot].residue_index, ri);
    }
}
```

The sequence rule is strict: separations 0 and 1 are rejected with the default value 2, while separation 2 is kept. The $N \cdots O$ source length rule also has exact boundaries: $L < 0.1 \text{ \AA}$ is rejected, $L = 0.1 \text{ \AA}$ is kept, $L > 50.0 \text{ \AA}$ is rejected, and $L = 50.0 \text{ \AA}$ is kept. A missing slot, missing backbone atom, duplicate atom pair, sequence rejection, or distance rejection creates no ResolvedHBond; there is no zero-valued source left behind. If the source list is empty, Compute() returns before the per-atom loop and the zero-initialized atom fields are what WriteFeatures() later writes.

2 Midpoint geometry

For accepted source k , midpoint is $\mathbf{m}_k = (\mathbf{n}_k + \mathbf{o}_k)/2$, distance is $L_k = |\mathbf{o}_k - \mathbf{n}_k|$, and $\mathbf{h_hat}$ is $\hat{\mathbf{h}}_k = (\mathbf{o}_k - \mathbf{n}_k)/L_k$. The direction is stored from donor N to acceptor O . The McConnell family is even under reversing this axis: $c = \hat{\mathbf{d}} \cdot \hat{\mathbf{h}}$ and $\hat{\mathbf{h}}$ both change sign, so the $c \hat{\mathbf{d}} \otimes \hat{\mathbf{h}}$ term keeps the same product and the $\hat{\mathbf{h}} \otimes \hat{\mathbf{h}}$ term is unchanged. The stored donor-to-acceptor sign is therefore a convention; it sets which way $\hat{\mathbf{h}}$ points but does not change the kernel value.

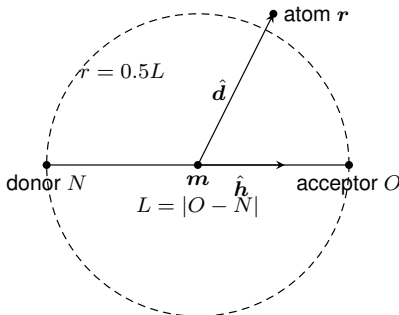


Figure 1: The source is the $N \cdots O$ segment reduced to its midpoint $\mathbf{m}$, length L , and axis $\hat{\mathbf{h}}$. The dashed circle marks the per-atom near-field boundary $r = 0.5L$; the atom path accepts only outside it.

For an atom at $\mathbf{r}_i$, the helper forms $\mathbf{d}$, $\mathbf{d}_{ik} = \mathbf{r}_i - \mathbf{m}_k$, its length r_{ik} , the unit viewing direction $\hat{\mathbf{d}}_{ik}$, and the dot product $c_{ik} = \hat{\mathbf{d}}_{ik} \cdot \hat{\mathbf{h}}_k$. This is the McConnell helper with $\hat{\mathbf{b}}$ replaced by $\hat{\mathbf{h}}$ and the expansion centre replaced by the hydrogen-bond midpoint. If $r_{ik} < 0.1 \text{ \AA}$, the helper returns the zero-initialized result with distance == 0; it does not clamp r to the guard distance.

```
// Distilled from ComputeHBondKernel.
Vec3 d = atom_pos - hbond_midpoint;          // midpoint -> atom
double r = d.norm();
if (r < cfg("singularity_guard_distance")) return result; // zero

result.distance = r;
double r3 = r * r * r;
Vec3 d_hat = d / r;
double c = d_hat.dot(h_hat);

result.f = (3.0 * c * c - 1.0) / r3;

for (int a = 0; a < 3; ++a)
  for (int b = 0; b < 3; ++b)
    result.M_over_r3(a,b) =
      (9.0*c*d_hat(a)*h_hat(b)
       -3.0*h_hat(a)*h_hat(b)
       -(3.0*d_hat(a)*d_hat(b) - (a == b ? 1.0 : 0.0))) / r3;
```

The scalar f is accumulated over the same accepted atom–source set as the tensor. Its relationship to the trace of the asymmetric M , and the way the three terms populate T_0 , T_1 , and T_2 , are the same as in the McConnell addendum after the substitution $\hat{\mathbf{b}} \mapsto \hat{\mathbf{h}}$.

3 Filters and accumulation

After the helper has produced a distance and candidate tensor, the atom path builds the evaluation context consumed by the filters. The filter set is minimum distance, self-source, then near-field. A rejected pair changes no accumulator: not the tensor sum, not `f_sum`, not the nearest-source record, and not the 3.5 Å count.

```
filters.Add(std::make_unique<MinDistanceFilter>());
filters.Add(std::make_unique<SelfSourceFilter>());
filters.Add(std::make_unique<DipolarNearFieldFilter>());

HBondKernelResult kernel =
    ComputeHBondKernel(atom_pos, hb.midpoint, hb.h_hat);

ctx.distance = kernel.distance;
ctx.source_extent = hb.distance;           // N...O length L
ctx.atom_index = ai;
ctx.source_atom_a = hb.donor_N;
ctx.source_atom_b = hb.acceptor_O;
ctx.sequence_separation = std::min(sep_to_donor, sep_to_acceptor);

if (!filters.AcceptAll(ctx)) continue;

if (kernel.distance < cfg("hbond_counting_radius")) ++nearby_count; // 3.5 Å
if (kernel.distance < nearest_dist) { nearest_dist = kernel.distance; ... }

M_total += kernel.M_over_r3;
f_sum += kernel.f;
```

`MinDistanceFilter` requires $r \geq 0.1 \text{ Å}$. For a helper return from below the guard, `kernel.distance` is zero and this filter omits the pair. `SelfSourceFilter` rejects the exact donor N and acceptor O atom indices. `DipolarNearFieldFilter` requires

$$r > \text{near_field_exclusion_ratio } L = 0.5L.$$

Equality at $0.5L$ is rejected on the atom path. The `sequence_separation` field is filled with the nearest endpoint-residue gap, but this filter set does not include `SequentialExclusionFilter`; sequence separation affects source resolution above, not later atom-source evaluation.

There is no spatial search cutoff in this path. `SpatialIndexResult` is a declared dependency, but after DSSP has supplied the source list the code loops over every resolved hydrogen bond for every atom. The only atom-source distance gates in the atom loop are the singularity and near-field filters; otherwise the kernel keeps its r^{-3} dependence for distant accepted terms. The count diagnostic uses accepted pairs only and tests $r < 3.5 \text{ Å}$, so equality at 3.5 Å is not counted.

4 Sampling path and outputs

`SampleKernelAt` evaluates the same stored source list at an arbitrary point. Because that point has no atom index, this path has no donor/acceptor self-source filter. It also uses an inline near-field test with a different boundary from the atom filter: it skips $r < 0.5L$, so equality at $0.5L$ is kept.

```
if (!conf_ || hbond_midpoints_.empty()) return SphericalTensor{};

Mat3 M_total = Mat3::Zero();
for (size_t hi = 0; hi < hbond_midpoints_.size(); ++hi) {
    auto kernel = ComputeHBondKernel(point, hbond_midpoints_[hi],
                                     hbond_directions_[hi]);
    if (kernel.distance < cfg("singularity_guard_distance")) continue;
    if (kernel.distance < cfg("near_field_exclusion_ratio") *
```

```

        hbond_distances_[hi]) continue;

    M_total += kernel.M_over_r3;
}
return SphericalTensor::Decompose(M_total);

```

For atoms, the stored tensor output is the direct decomposition of the raw asymmetric sum $M_i = \sum_k M^{(ik)}$. There is no symmetrization and no trace subtraction before `SphericalTensor::Decompose`. The nearest source is tracked only among accepted pairs; a local 99 Å sentinel initializes that search, but if no source is accepted the atom's written nearest-distance and inverse-cube fields remain their zero defaults.

```

if (nearest_hb_idx != SIZE_MAX) {
    ca.hbond_nearest_dist = nearest_dist;
    ca.hbond_nearest_dir =
        (atom_pos - nearest_hb.midpoint).normalized();
    ca.hbond_inv_d3 = 1.0 / (nearest_dist * nearest_dist * nearest_dist);

    auto nearest_kernel =
        ComputeHBondKernel(atom_pos, nearest_hb.midpoint, nearest_hb.h_hat);
    ca.hbond_nearest_spherical =
        SphericalTensor::Decompose(nearest_kernel.M_over_r3);
}

ca.hbond_shielding_contribution =
    SphericalTensor::Decompose(M_total);    // raw M, T0 + T1 + T2
ca.hbond_mcconnell_scalar = f_sum;

```

The writer emits two arrays. `hbond_shielding.npy` is $(N, 9)$ in the shared packed order

$$[T_0, T_1[0..2], T_2[0..4]]$$

written by `PackFull19`. It is the full asymmetric M decomposition. `hbond_scalars.npy` is $(N, 4)$: nearest atom-to-midpoint distance, the corresponding $1/r^3$ field, accepted-source count with $r < 3.5$ Å, and the accepted $\sum f$. HBond writes no file analogous to McConnell's category T_2 arrays. No HBond output first takes the symmetric-traceless projection alone; the symmetric-traceless part is present only as the T_2 channel inside the full decomposition.

```

ca.hbond_shielding_contribution.PackFull19(&shielding[i*9]);

scalars[i*4 + 0] = ca.hbond_nearest_dist;
scalars[i*4 + 1] = ca.hbond_inv_d3;
scalars[i*4 + 2] = double(ca.hbond_count_within_3_5A);
scalars[i*4 + 3] = ca.hbond_mcconnell_scalar;

WriteFloat64("hbond_shielding.npy", shielding.data(), N, 9);
WriteFloat64("hbond_scalars.npy", scalars.data(), N, 4);

```

5 Glossary

DSSP partner slot. One residue pointing at another it hydrogen-bonds to (a per-residue DSSP field holding a partner residue index, with the actual N and O atoms still to be looked up). If residue i has acceptor slot j , HBond turns it into $N_i \rightarrow O_j$; if residue i has donor slot j , it turns it into $N_j \rightarrow O_i$. Formally, each slot contains a residue index or `SIZE_MAX`, and HBond maps a valid slot to an ordered backbone donor–acceptor atom pair.

Hydrogen-bond source segment. The source is the $N \cdots O$ line collapsed to a midpoint, a length, and an axis. A donor N at $(0, 0, 0)$ and acceptor O at $(3, 0, 0)$ produce midpoint $(1.5, 0, 0)$, length 3 Å, and axis $(1, 0, 0)$. Formally, $m = (n + o)/2$, $L = |o - n|$, and $\hat{h} = (o - n)/L$.

Near-field exclusion. A blind spot around the $N \cdots O$ segment, where the atom sits too close for the midpoint to stand in for the two-point source (the atom path drops any atom whose distance to the midpoint is at or below $\rho_{\text{nf}}L$, the ball of radius half the $N \cdots O$ length when $\rho_{\text{nf}} = 0.5$). For an $N \cdots O$ length of $3 \text{ \AA}$, the atom path omits an evaluation at $r = 1.5 \text{ \AA}$ and accepts one outside that distance. Formally, the atom filter is $r > \rho_{\text{nf}}L$, with $\rho_{\text{nf}} = \text{near_field_exclusion_ratio} = 0.5$.

Spherical tensor split. A 3×3 tensor holds three kinds of content that rotate without mixing — an orientation-free size (the scalar trace, T_0), an axial twist from its antisymmetric part (the three T_1 components), and a five-number traceless shape for the rest of the direction-dependence (the five T_2 components). In HBond, `hbond_shielding.npy` keeps all nine numbers from the raw asymmetric McConnell-family sum, while no written HBond array keeps only the symmetric-traceless projection. Formally, it is the closed-form split of a 3×3 tensor's nine components into dimensions $1 + 3 + 5$, with the T_2 basis normalized to preserve the Frobenius norm.